

MANUAL

Setup Wizard — Reference Manual

Precise reference for `scripts/setup-agents-wizard.sh`. Every statement here was read out of the script or verified against the installed CLI. For a walkthrough aimed at getting set up, read [USER_GUIDE.md](#) instead.

`scripts/setup-agents-wizard.sh` is the single source of truth. This manual follows it. (An earlier revision of [README.md](#) embedded a stale copy of the script; that listing has since been removed.)

Synopsis

`scripts/setup-agents-wizard.sh`

No arguments. No options. Non-interactive apart from **one** blocking read at the very end, which waits for Enter after the [ACTION_REQUIRED](#) section; it is skipped when stdin is not a terminal or when `WIZARD_NONINTERACTIVE` is set. The only other input it can ask for is a sudo password, when installing jq, installing Ollama, or enabling the ollama service — and that prompt comes from sudo, not from the script. Behaviour is tuned entirely through the environment: `WIZARD_INDEX_PROJECT`, `WIZARD_KEEP_TELEMETRY`, `WIZARD_SKIP_GLYPHDOWN_HOOK`, `WIZARD_NONINTERACTIVE` and `WIZARD_STATE_DIR` are the five switches — see [Environment variable reference](#).

Three companion scripts handle post-installation operations and are documented separately in [OPERATIONAL-SCRIPTS.md](#):

```
scripts/ollama-vulkan-remediation.sh [--check|--apply|--verify|--rollback|--help]
scripts/lumen-reindex.sh                [project-path] [--force] [--
```

```
allow-gpu]
scripts/lumen-index-doctor.sh          [project-path]
scripts/audit-hardcoded-paths.sh       [repo-path] [--list|--help]
```

The wizard **never invokes them**. It names the first three in ACTION REQUIRED when its own read-only probes say they are needed; the fourth is a repository hygiene check, guarded by test group J.

```
SETUP_WIZARD_LIB_ONLY=1 . scripts/setup-agents-wizard.sh
```

Library mode: source the helper functions without executing the wizard. See [Library mode](#).

```
scripts/rollback-agents-wizard.sh [--list] [--session <id|latest>]
[--component <name>]...
                                [--dry-run] [--yes] [--run-
actions] [--help]
```

The companion undo tool. It replays the manifest this wizard writes. Full reference: [SAFETY-AND-ROLLBACK.md](#).

The script runs under `set -euo pipefail`. It resolves its own location and derives the project root as the parent of its directory:

Variable	Value
SCRIPT_DIR	<code>cd -- "\$(dirname -- "\${BASH_SOURCE[0]}")" && pwd</code> — absolute path of scripts/
PROJECT_ROOT	<code>dirname "\$SCRIPT_DIR"</code> — the repository root

Your working directory at launch is irrelevant; the wizard cds to PROJECT_ROOT in Step 6.

Execution order

Step	Header	Work performed
—	AI Agents Ultimate Setup Wizard	<code>backup_init</code> — opens the rollback session before Step 1 , so every later mutation is recorded
1	Step 1: Checking System Prerequisites	Verifies git, curl, node, npm; calls <code>ensure_jq</code>
2		

Step	Header	Work performed
		Bun; npm_install_if_missing for codegraph and glyphdown; specify via uv tool install specify-cli;
	Step 2: Installing / Updating Global CLI Tools	npm_install_if_missing qwen; detection-only roll call for Kimi / Opencode / MiMo Code; ashlr; WOZCODE
3	Step 3: Lumen Semantic Search Setup	ensure_lumen, which begins with configure_telemetry_optout
4	Step 4: Verifying CLI Availability	Roll call over 8 commands: lumen codegraph glyphdown specify kimi opencode mimo qwen. ashlr is not among them — it is a plugin with no binary
5	Step 5: Configuring MCP Servers for Each Agent	claude mcp add-json for Claude Code, a jq mirror of the same entry into ~/.claude.json, 2 plugin clones, Glyphdown PreToolUse/PostToolUse hooks (unless WIZARD_SKIP_GLYPHDOWN_HOOK), then ~/.kimi-code/mcp.json, ~/.config/opencode/ opencode.json and ~/.qwen/settings.json; MiMo Code is probed with timeout 25 mimo mcp list, not written to
6	Step 6: Project-Level Setup (root = ...)	submodules/, SpecKit init, setup_superspec, extension registration
7	Step 7: Ollama Concurrency Tuning (host-specific)	tune_ollama_step, which delegates to scripts/ollama-tune.sh (override the path with OLLAMA_TUNE_SCRIPT). Report-only by default: it prints the delegate's detection and its --print-

Step	Header	Work performed
		commands output for this host, then registers those exact commands as a manual step. --apply is reached only when WIZARD_TUNE_OLLAMA is set and no indexer is in flight and the operator confirms (or WIZARD_NONINTERACTIVE is set); applying restarts ollama and aborts in-flight embedding requests. The delegate's exit code is a three-valued verdict (0 fine, 1 action required, 2 could not determine) and all three are preserved — 1 is the case that HAS a recommendation, not a failure. Absent script, absent ollama, a timeout, or a 2: reported and skipped, never a fabricated value Opt-in, WIZARD_INDEX_PROJECT only. codegraph sync "\$PROJECT_ROOT" when \$PROJECT_ROOT/.codegraph/codegraph.db exists, else codegraph init "\$PROJECT_ROOT"; then lumen index "\$PROJECT_ROOT". Each half is skipped with a warning if its CLI is missing. Unset, the header never prints and the wizard emits Skipping project indexing (set WIZARD_INDEX_PROJECT=1 to build indexes). verify_provider_ci_step, which delegates to scripts/verify-provider-ci.sh (override with
8	Step 8: Indexing This Project	
9	Step 9: Provider-Side CI Verification	

Step	Header	Work performed
		PROVIDER_CI_SCRIPT). Read-only. Its three-valued verdict is preserved exactly: 0 nothing found → success line; 1 confirmed → a manual step carrying the verifier's own findings; 2 or any other status → UNVERIFIED, never rendered as the 0 success line . Absent script or absent gh: also UNVERIFIED, and the wizard continues
—	Setup Complete – Final Summary	Six / sections — Installed Global Commands, Claude Code plugins / optional tools, Agent MCP Configurations (Lumen), Lumen Semantic Search, Telemetry / analytics, Project
§9	ACTION REQUIRED – N step(s) only you can do	The manual-step registry is drained: pending steps are printed, written to \$PROJECT_ROOT/MANUAL-STEPS.md, and on a TTY the script blocks on read until Enter. See <u>Step 9</u>

The §9 in the last row is the **source comment's** section number, not a printed banner — and it is now distinct from the printed Step 9: Provider-Side CI Verification. The wizard prints numbered Step N: banners for 1–9; A10 asserts those literals read exactly 1,2,3,4,5,6,7,8,9,.

Step 9 — ACTION REQUIRED (manual steps)

Steps a shell script genuinely cannot perform: Claude Code slash commands, and privileged host changes needing the operator's `sudo`. Rather than skipping them silently — which is how a wizard ends up reporting success for work that was never done — the wizard **detects the pending state** and hands over the exact commands.

The registry is two parallel arrays plus one helper, defined near the top of the file:

```
MANUAL_STEPS=(); MANUAL_TITLES=()
manual_step() { MANUAL_TITLES+=("$1"); MANUAL_STEPS+=("$2"); }
CLAUDE_DIR="${CLAUDE_CONFIG_DIR:-$HOME/.claude}"
```

Conditions, in emission order:

#	Raised when	Step
1	<code>\$CLAUDE_DIR/plugins/cache/ashlr-marketplace/ashlr/</code> exists and <code>\$CLAUDE_DIR/plugins/marketplaces/ashlr-marketplace/</code> does not	Activate the ashlr plugin (<code>/plugin marketplace add,/plugin install,/reload-plugins,/ashlr:ashlr-status</code>)
2	<i>(elif)</i> the marketplace dir exists and <code>\$PROJECT_ROOT/.ashlrcode/genome</code> does not	<code>/ashlr:ashlr-genome-init</code>
3	ollama on PATH and its journal (scanned from <code>systemctl show ollama -p ActiveEnterTimestamp</code>) reports library=Vulkan/vulkan	<code>./scripts/ollama-vulkan-remediation.sh --check then --apply</code>
4	wozcode missing and <code>WOZCODE_INSTALL_CMD</code> unset	Supply <code>WOZCODE_INSTALL_CMD</code>
5	<code>WIZARD_SKIP_GLYPHDOWN_HOOK</code> set	Re-run without the flag
6	lumen ON PATH and timeout 20 <code>lumen search "x" -p "\$PROJECT_ROOT" --summary -n 1</code> does not succeed	<code>./scripts/lumen-reindex.sh then ./scripts/lumen-index-doctor.sh</code>

#	Raised when	Step
7	always	<code>exec bash -l</code>

The cache/ vs marketplaces/ distinction is the mechanism. `plugins/cache/<mkt>/` means the vendor installer cloned the bits; `plugins/marketplaces/<mkt>/` is written by Claude Code itself when a human runs `/plugin marketplace add`. Only the second means the plugin is wired in. Detection is therefore stateful and self-clearing — do the step, re-run, and the entry disappears, with no checklist to maintain. Test **A50** asserts the `marketplaces/` probe specifically.

Note the inconsistency in the current source: this block honours `CLAUDE_CONFIG_DIR`, while Step 2's install check and the final summary both hardcode `$HOME/.claude/plugins/cache/...`. On a non-default config dir they can disagree; the `ACTION REQUIRED` block is the one reading the active directory.

Timeouts here are load-bearing, not decoration. The `journalctl/systemctl` probes are wrapped in `timeout 20 / timeout 10`, and the `lumen search` probe in `timeout 20` — an unbounded version once stalled the wizard for ten minutes while an index run held the backend. A timeout is treated as “*could not confirm*” and **raises** the step rather than skipping it. Test **A47** counts bounded against unbounded `lumen search` occurrences and requires them equal.

Output:

```
print_header "ACTION REQUIRED – ${#MANUAL_TITLES[@]} step(s) only
you can do"
```

followed by the two yellow disclaimer lines (A shell script cannot run Claude Code slash commands or use your `sudo`. / These are NOT done. Nothing above claims they are.), the numbered list indented five spaces, and the path to `MANUAL-STEPS.md`. Then:

```
if [[ -t 0 && -z "${WIZARD_NONINTERACTIVE:-}" && $
    {#MANUAL_TITLES[@]} -gt 0 ]]; then
    read -r -p "Read the N step(s) above. Press Enter to
    continue..." _ack || true
fi
```

All three conditions must hold. Because step 7 is unconditional the array is never empty, so in practice the pause fires on any TTY run without `WIZARD_NONINTERACTIVE`. The `|| true` keeps an EOF on stdin from aborting under `set -euo pipefail`. Tests **A48** and **A49**.

Full narrative: [ACTION-REQUIRED.md](#).

Exit codes

Code	Condition
0	The wizard reached its terminal exit 0. Warnings may still have been printed — a 0 means “ran to completion”, not “everything succeeded”. Library mode also returns/exits 0.
1	One or more of git, curl, node, npm is missing (Step 1 aborts before touching anything).
1	ensure_jq could not install jq: no apt-get/yum on Linux, an \$OSTYPE that is neither linux-gnu* nor darwin*, or jq still absent after the install attempt.
other	Any unguarded command failing under set -e propagates its status. Every network install, clone and optional CLI is wrapped in <code>\ \ true</code> or an explicit warning, so this is rare in practice.

There is no separate exit code for “finished with warnings”. Detect that by grepping the output for `/` , or by running `scripts/test-setup-agents-wizard.sh`, which does exit non-zero on failure.

The exit code also says nothing about the ACTION REQUIRED steps: a run that ends 0 with five outstanding manual steps is normal and expected. Parse `$PROJECT_ROOT/MANUAL-STEPS.md` if you need that programmatically — it lists exactly what is still pending, regenerated each run.

The three operational scripts use their own schemes; `lumen-index-doctor.sh` in particular returns **1 for “corruption found”**, so it is usable as a gate. See [OPERATIONAL-SCRIPTS.md](#):

Script	0	1	2	3	4
ollama-vulkan-remediation.sh	success (and always for --check)	--verify probe failed; write/restart failed	bad option	—	—
lumen-reindex.sh	index completed	gave up after MAX_ROUNDS	—	refused: library=Vulkan	batch probe still failing
lumen-index-doctor.sh	healthy	corruption found	could not inspect	—	—
audit-hardcoded-paths.sh	no machine-specific paths (also --list)	violations found, or cd "\$ROOT" failed	the target directory does not exist	—	—

scripts/rollback-agents-wizard.sh uses a different scheme:

Code	Condition
0	Rollback applied with no failures. Also --list, --help, --dry-run, “nothing to roll back for the selected components”, and declining the prompt.
1	At least one entry failed — a missing backup file, or a cp/rm that did not succeed. Also: no backup root, or no manifest.tsv for the chosen session. A failed ACTION undo command only warns and does not set this.
2	An unrecognised option.

Function reference

Everything above the `SETUP_WIZARD_LIB_ONLY` guard is pure definitions and is safe to source. `$1`, `$2` below are positional parameters. One helper — `npm_install_if_missing` — is defined *below* the guard and is therefore not available in library mode.

Output and probing

Function	Arguments	Purpose	Side effects
<code>print_header</code>	<code>\$1</code> = title	Prints a cyan ==== banner: blank line, rule, title, rule	stdout only
<code>print_success</code>	<code>\$1</code> = message	message in green	stdout only
<code>print_error</code>	<code>\$1</code> = message	message in red	stdout only — does not exit ; the caller decides
<code>print_warning</code>	<code>\$1</code> = message	message in yellow	stdout only
<code>print_info</code>	<code>\$1</code> = message	message in blue	stdout only
<code>check_command</code>	<code>\$1</code> = command name	<code>command -v "\$1" >/dev/ null 2>&1</code>	None. Returns 0 if found, 1 otherwise. Prints nothing.

All five printers write to **stdout**, not `stderr` — redirecting `2>` will not separate errors from normal output.

Manual-step registry

Name	Kind	Purpose
<code>MANUAL_TITLES</code>	array	One short title per pending step
<code>MANUAL_STEPS</code>	array	The command block for the step at the same index

Name	Kind	Purpose
manual_step	function, \$1 = title, \$2 = command block	Appends to both arrays. No output of its own; the registry is drained once, at the end of the run
CLAUDE_DIR	constant	\$ {CLAUDE_CONFIG_DIR:-\$HOME/.claude} — the active Claude Code config dir, which is where claude mcp and /plugin actually write

Available in library mode: all four are defined above the `SETUP_WIZARD_LIB_ONLY` guard, so a test can call `manual_step` and inspect the arrays without running the wizard. Nothing consumes them until [Step 9](#).

The \$2 block is stored verbatim, newlines included, and re-emitted two ways: indented five spaces on the console (while `IFS= read -r line; do echo " $line"; done`), and inside a fenced code block in `MANUAL-STEPS.md`. Comment lines beginning # inside a block are part of the step text, not shell comments the wizard evaluates.

Backup and rollback

These four are the transactional layer. Everything they record is replayed in reverse by `scripts/rollback-agents-wizard.sh`; see [SAFETY-AND-ROLLBACK.md](#) for the manifest format and the rollback CLI.

Function	Arguments	Purpose	Side effects
backup_init	none	Opens a rollback session for this run	Re-resolves WIZARD_STATE_DIR at call time , not at source time, so exporting it after sourcing the wizard still works. Sets <code>BACKUP_ROOT</code> , <code>BACKUP_SESSION="\$BACKUP_ROOT/\$(date -u +%Y%m%dT%H%M%S)"</code> and <code>MANIFEST</code> ; <code>mkdir -p "\$BACKUP_SESSION/files"</code> ; writes the TSV header row; <code>ln</code>

Function	Arguments	Purpose	Side effects
snapshot_before	\$1 = component, \$2 = target path	Records the pre-change state of one path	-sfn "\$BACKUP_SESSION" "\$BACKUP_ROOT/latest". Prints the session path and the rollback command. Called once, immediately before Step 1. No-op (returns 0) when \$MANIFEST is empty — i.e. when backup_init never ran. Returns 0 without a second row if \$2 already appears in column 3 of this session's manifest, so the FIRST snapshot wins and a file touched by several steps still restores to its true original. If the target exists: action MODIFIED, cp -p into \$BACKUP_SESSION/files/<first-16-hex-of-sha256(path)>__<basename>, sha256_before from _sha. A failed copy prints Could not back up ..., returns 1 and writes no row. If the target does not exist: action CREATED, backup and sha256_before both -. Appends one tab-separated row.
record_action	\$1 = component, \$2 = undo command	Records a change that is not a file	No-op when \$MANIFEST is empty. Appends a row with action ACTION, \$2 in the target column, - for backup and sha256_before. The rollback tool prints these and runs them only with --run-actions.
_sha	\$1 = path	Digest helper	sha256sum "\$1" \ cut -d" " -f1 when \$1 is a regular file, otherwise the literal -. Prints to stdout; no side effects.

Components in use: lumen, shell, claude, kimi, opencode, qwen, npm, speckit, and misc as the unused fallback. See [SAFETY-AND-ROLLBACK.md → Components](#).

Files and JSON

Function	Arguments	Purpose	Side effects
backup_file	\$1 = path, \$2 = component (optional, default misc)	Sibling copy and manifest registration before modification	Always calls snapshot_before "\$2" "\$1" first — so even a file that does not yet exist is recorded (as CREATED). Then, if \$1 is a regular file, cp -p "\$1" "\$1.bak.\${date +%Y%m%d%H%M%S}" and prints Backed up \$1. The .bak half is a silent no-op when the file does not exist; the manifest half is not. cp -p preserves the mode.
ensure_jq	none	Guarantees jq is available	Returns immediately if jq is present. Linux: sudo apt-get update -qq && sudo apt-get install -y jq, else sudo yum install -y jq, else exit 1. macOS: brew install jq, installing Homebrew via the official curl

Function	Arguments	Purpose	Side effects
configure_mcp_for_agent	\$1 = config file path, \$2 = agent display name, \$3 = component (optional, default misc)	Merges the Lumen and CodeGraph MCP servers into a .mcpServers config	script first if brew is absent. Any other \$OSTYPE: exit 1. Re-checks afterwards and exit 1s if still missing. mkdir -p "\$(dirname "\$1")"; snapshot_before "\$3" "\$1"; writes {} if the file does not exist; backup_file "\$1" "\$3"; mktemp; runs jq --arg lumen_bin "\$LUMEN_WRAPPER" '.mcpServers += {...}' and mv's the temp file over the original. Prints one line. The += merge preserves every other key. Used for Kimi and Qwen Code only — Claude Code goes through claude mcp, and Opencode has a different schema. The snapshot is taken <i>before</i> the {} seed, so a config the wizard creates from scratch is recorded CREATED and rollback deletes it rather

Function	Arguments	Purpose	Side effects
			than leaving an empty {} behind. Tests G16 and G17 are the regression guards.
			Returns 0 if the file is absent or the start marker is not present. Otherwise awk writes "\$1.lumen.tmp", cats it back over \$1 (preserving the file's mode and inode), then removes the temp file. Uses awk, not sed, because the markers contain regex metacharacters. Lines are matched with index(\$0, marker) == 1, i.e. the marker must start at column 1.
strip_managed_block	\$1 = file, \$2 = start marker, \$3 = end marker	Removes a marker-delimited block	

Defined inside Step 2, not above the library guard

Function	Arguments	Purpose	Side effects
npm_install_if_missing	\$1 = command name, \$2 = npm package	Installs a global npm package only when its command is missing	Returns 0 with \$1 already present - leaving the existing install alone. when check_command

Function	Arguments	Purpose	Side effects
			"\$1" succeeds — a pre-existing install is never touched or upgraded. Otherwise runs <code>npm install -g "\$2" >/dev/null 2>&1</code> and re-checks the command. On success: <code>record_action npm "npm uninstall -g \$2" and \$1 installed from \$2..</code> On failure: <code>Could not install \$2 - '\$1' stays unavailable.</code> and no manifest row, so rollback never uninstalls something the wizard did not install.

This one is defined between the step headers rather than in the definition block at the top of the file, so it is **not** available in library mode. Call sites: `npm_install_if_missing codegraph "@colbymchenry/codegraph", npm_install_if_missing glyphdown "glyphdown", npm_install_if_missing qwen "@qwen-code/qwen-code".`

Note the third one: the package is `@qwen-code/qwen-code`, but the **command it installs is `qwen`**. Probing for a command named `qwen-code` reported a false “install failed” for software that was already working, so test **A22** asserts that `npm_install_if_missing` is never called with `qwen-code` as its first argument.

SpecKit does not go through it — `specify` is a Python tool installed with `uv tool install specify-cli`, and that branch records its own undo row, `record_action speckit "uv tool uninstall specify-cli"` (test **A28**).

The MCP block `configure_mcp_for_agent` writes, with `$LUMEN_WRAPPER` expanded:

```
{
  "mcpServers": {
    "lumen": {
      "command": "/home/<you>/.local/bin/lumen",
      "args": ["stdio"]
    },
    "codegraph": {
      "command": "codegraph",
      "args": ["serve"]
    }
  }
}
```

The Lumen command is an **absolute path** on purpose: agents spawn MCP servers from non-interactive shells where `~/.local/bin` is frequently absent from `PATH`, so a bare `"lumen"` would fail to launch. `stdio` is Lumen’s real MCP subcommand — `lumen serve` does not exist and fails with `Error: unknown command "serve" for "lumen"`.

Lumen setup

Function	Arguments	Purpose	Side effects
<code>install_lumen_wrapper</code>	<code>none</code>	Writes the version-agnostic lumen launcher	<code>mkdir -p ~/.local/bin;</code> <code>backup_file</code> <code>"\$LUMEN_WRAPPER" lumen;</code> writes the wrapper from a quoted heredoc; <code>chmod 755</code> . Prints Lumen launcher installed:
<code>install_lumen_completion</code>	<code>none</code>	Installs bash completions	Warns and returns 0 if the wrapper is not executable. <code>mkdir -p ~/.local/share/</code>

Function	Arguments	Purpose	Side effects
configure_lumen_shell	none	Wires ~/.local/ bin onto PATH for both shell kinds	bash-completion/ completions; runs "\$LUMEN_WRAPPER" completion bash into .../ completions/lumen. On failure, deletes the (empty) file and warns rather than leaving a broken completion script behind. For ~/.bashrc: backup_file, strip_managed_block with the Lumen markers, append a fresh block. For ~/.bash_profile: backup_file, strip_managed_block with the user-bin markers, append a fresh block. Finally export PATH="\$HOME/.local/bin: \$PATH" in the <i>running</i> process so later steps see it. Prints two lines. Installs Ollama if absent — Linux: curl -fsSL https:// ollama.com/install.sh \\ sh; macOS with Homebrew: brew install ollama. Returns 1 if ollama is still not on PATH (two warnings printed). If systemctl exists and the unit is inactive: sudo systemctl enable -- now ollama, then reports active/not-active. Finally, if ollama list does not contain \$LUMEN_EMBED_MODEL, runs ollama pull "\$LUMEN_EMBED_MODEL" (~320 MB, one-time). A failed pull is a warning, not an error.
ensure_ollama	none	Guarantees a reachable embedding backend	
verify_lumen	none		

Function	Arguments	Purpose	Side effects
		Post-install health check	Three probes, no mutations: (1) "\$LUMEN_WRAPPER" version runs — otherwise and rc=1; (2) bash -lc 'command -v lumen' succeeds — otherwise only, no rc change; (3) the embedding backend, in two stages — see below. Returns \$rc. Returns 0 immediately with WIZARD_KEEP_TELEMETRY set - leaving telemetry settings alone. when WIZARD_KEEP_TELEMETRY is non-empty. Otherwise: backup_file "\$HOME/.bashrc" shell; strip_managed_block with the privacy markers; appends a block exporting DO_NOT_TRACK=1 and CODEGRAPH_TELEMETRY=0; exports both into the running process. Then, when codegraph is on PATH, codegraph telemetry off (a failure warns only). Finally, when ~/.qwen/ . settings.json exists and jq is available, reads .usageStatisticsEnabled \\\n tostring — if it is already "false" it prints Qwen usage statistics already disabled., otherwise backup_file "\$qwen_cfg" qwen and jq '.usageStatisticsEnabled = false' via mktemp + mv, skipping the write if jq
configure_telemetry_optout	none	Turns usage analytics off across the installed tooling	

Function	Arguments	Purpose	Side effects
ensure_lumen	none	Step 3 in one call	produced an empty file. See Managed blocks. Runs <pre> configure_telemetry_optout, install_lumen_wrapper, install_lumen_completion, configure_lumen_shell, ensure_ollama \\ \\ true, verify_lumen \\ \\ print_warning "Lumen setup finished with warnings (see above)". Always returns 0 — a Lumen problem never aborts the wizard.</pre>

The backend probe in verify_lumen

Reachability is not health. /api/tags answers 200 while embedding is completely broken: under sustained load the ollama runner can wedge into a state where /api/embed returns HTTP 500 with {"error": "failed to encode response: json: unsupported value: NaN"} for *every* input. Lumen then aborts and prints its usage text after the error, which reads like CLI misuse rather than a backend fault. So the wizard proves the backend with a real round-trip:

Stage	Probe	Outcome
1	curl -sf --max-time 5 "\$host/api/tags"	Fails → Embedding backend unreachable at <host> - Lumen cannot index., rc=1, stage 2 is skipped Response contains "embeddings" → Embedding backend
2	curl -s --max-time 90 "\$host/api/embed" -d '{"model": "\$LUMEN_EMBED_MODEL", "input": "health check"}'	

Stage	Probe	Outcome
		healthy at <host> (round-trip returned a vector). Response contains NaN → Embedding backend is WEDGED at <host> - it returns NaN for every input. plus Fix: ollama stop <model> (a fresh runner clears it), rc=1 Anything else → Embedding round-trip failed at <host> - Lumen will fail to index., rc=1
2	same	
2	same	

\$host is \${OLLAMA_HOST:-http://localhost:11434}. The 90-second timeout on stage 2 is generous on purpose: the first embed after a cold start has to load the model. Tests **A33** and **A34** assert that the health check uses /api/embed and that the wedged-NaN case is detected by name.

Project setup

Function	Arguments	Purpose	Side effects
setup_superspec			

Function	Arguments	Purpose	Side effects
	\$1 = project root, \$2 = submodule path (relative)	Safely obtains the SuperSpec checkout	Four branches, in order. (a) \$2/.git exists → already present; reports “git submodule” if it is a file, “standalone clone” if a directory; nothing is deleted. (b) git -C "\$1" submodule status "\$2" succeeds → git submodule update --init --recursive "\$2". (c) \$2 exists but is not a git checkout → rm -rf "\$2" after printing Removing stale non-git directory at (d) clones https://github.com/WangX0111/superspec.git into \$2.

The -e test in branch (a) is the fix for a data-loss bug: for a **registered git submodule**, <path>/.git is a **file** (a gitlink), never a directory. The previous -d test was therefore always false for a submodule checkout, so the rm -rf below it deleted the registered submodule on every run and replaced it with a plain clone, corrupting the parent repository’s

submodule state. scripts/test-setup-agents-wizard.sh test **E1** is the regression guard: it builds a fake gitlink with a canary file and asserts the canary survives.

Constants

Name	Value
WIZARD_STATE_DIR	<code>\${WIZARD_STATE_DIR:-\$HOME/.local/share/setup-agents-wizard}</code> — respects a pre-set value, and is re-resolved inside backup_init
BACKUP_ROOT	<code>\$WIZARD_STATE_DIR/backups</code>
BACKUP_SESSION	<code>\$BACKUP_ROOT/\$(date -u +%Y%m%dT%H%M%S)</code> — empty until backup_init runs
MANIFEST	<code>\$BACKUP_SESSION/manifest.tsv</code> — empty until backup_init runs, which makes snapshot_before and record_action no-ops
LUMEN_WRAPPER	<code>\$HOME/.local/bin/lumen</code>
LUMEN_COMPLETION_DIR	<code>\$HOME/.local/share/bash-completion/completions</code>
LUMEN_EMBED_MODEL	<code>\${LUMEN_EMBED_MODEL:-ordis/jina-embeddings-v2-base-code}</code> — respects a pre-set value
LUMEN_BLOCK_START / LUMEN_BLOCK_END	<code>~/.bashrc</code> markers for the Lumen block (see below)
USERBIN_BLOCK_START / USERBIN_BLOCK_END	<code>~/.bash_profile</code> markers
PRIVACY_BLOCK_START / PRIVACY_BLOCK_END	<code>~/.bashrc</code> markers for the telemetry opt-out block. Defined <i>below</i> <code>configure_lumen_shell</code> , immediately above <code>configure_telemetry_optout</code>
SUPERSPEC_PATH	<code>submodules/superspec</code> , relative to <code>PROJECT_ROOT</code>

The lumen wrapper

`~/local/bin/lumen` is a generated bash script, not a symlink. It exists because the plugin ships its binary at a **version-pinned** path:

```
~/claude-shared/plugins/cache/<marketplace>/lumen/<version>/bin/  
lumen-<os>-<arch>
```

A symlink into that path breaks the moment the plugin updates, because the old version directory is removed. The wrapper resolves the newest installed version at *call* time.

Resolution order:

1. **LUMEN_BIN override.** If set, the wrapper execs it. If it is set but not executable, the wrapper prints `lumen: LUMEN_BIN is set but not executable: ...` to stderr and exits **127**.
2. **Host mapping.** `uname -s` → `linux` / `darwin`; `uname -m` → `amd64` (`x86_64`, `amd64`) / `arm64` (`aarch64`, `arm64`). Anything else exits **127**.
3. **Primary glob:** `~/claude-shared/plugins/cache/*/lumen/*/bin/lumen-<os>-<arch>`.
4. **Fallback glob:** `~/claude*/plugins/cache/*/lumen/*/bin/lumen-<os>-<arch>` — catches per-account Claude config directories carrying their own plugin cache.
5. Among the executable candidates, the version component is extracted from the path (`${f%/bin/*}` then `${f##*/}`) and the highest is chosen with **sort -V**. Version sort, not lexical: `0.0.100` must beat `0.0.41` and `0.0.9`. Test **B3** asserts exactly this.
6. If nothing matched, it prints a multi-line diagnostic naming both search paths, pointing at `/plugin` and at `LUMEN_BIN`, and exits **127** — never 0. Test **B8** asserts the non-zero exit so a missing plugin cannot look like success.

The wrapper runs under `set -uo pipefail` (no `-e`) and `shopt -s nullglob`. It is disposable: delete it and re-run the wizard, or re-create it, to restore `lumen` on `PATH`.

Files created and modified

Everything below is created or modified by the wizard. `<ts>` is `YYYYMMDDHHMMSS` (local time), and `<utc>` is `YYYYMMDDTHHMMSSZ`. The **Component** column is the name `scripts/rollback-agents-wizard.sh` — `--component` filters on.

Home directory

Path	Component	Action	Detail
~/.local/share/setup-agents-wizard/backups/<utc>/	—	Created	The rollback session for this run: manifest.tsv plus files/, and a later symlink alongside it. Override the roll WIZARD_STATE_DIR.
~/.local/bin/lumen	lumen	Created / overwritten	The version-agnostic launcher, mode Previous copy saved as ~/.local/bin/lumen.bak.<ts>.
~/.local/share/bash-completion/completions/lumen	lumen	Created / overwritten	Output of lumen completion bash. La by bash-completion, so it costs nothing shell startup. Deleted instead of left generation fails. Registered with snapshot_before, so it gets no .bak.< sibling.
~/.bashrc	shell	Appended (two managed blocks)	Backed up to ~/.bashrc.bak.<ts> — run , because configure_telemetry_or configure_lumen_shell each call back The manifest still holds a single row (first snapshot wins). Each block is st before a fresh copy is appended: the telemetry opt-out block first, then the block. Content outside the markers is touched.
~/.bash_profile	shell	Appended (managed block)	Backed up to ~/.bash_profile.bak.< strip-then-append treatment with the bin markers.
(active CLAUDE_CONFIG_DIR)	claude	Modified by the claude CLI, never by the wizard	claude mcp add-json lumen '{"command": "<wrapper>", "args": ["s -s user, skipped when claude mcp g already succeeds. Undo is recorded a ACTION: claude mcp remove lumen -s user writes into \$CLAUDE_CONFIG_DIR ~/.claude), which is not necessarily de.json — hence the row below.
~/.claude.json	claude	Merged with jq	Because of the row above. .mcpServe = {"type": "stdio", "command": <wrapper ["stdio"]}, written only when the file already exists and jq is available, and skipped with a success line when

Path	Component	Action	Detail
~/.claude/ settings.json	claude	Created / merged	.mcpServers.lumen is already present backup_file ... claude runs first, so it recorded MODIFIED and rollback resto byte-exactly; a .bak.<ts> sibling is w too. Tools that inherit from this file – Code reports its servers as claude:~/.claude.json — would othe never see Lumen. Test A44. Written only when WIZARD_SKIP_GLYPHDOWN_HOOK is unset glyphdown is on PATH — a hook for a m binary would fire and fail on every t and the opt-out lets you install glyph without wiring it in. snapshot_before runs first, then {} if absent, then .hooks.PreToolUse and .hooks.PostTo each gain {"matcher":"*","hooks": [{"type":"command","command":"glyph guarded by an index("glyphdown") cl re-runs do not duplicate and existing survive. mktemp + mv, and the write is entirely if jq produced an empty file up to ~/.claude/settings.json.bak.< Because the snapshot precedes the { a clean machine the manifest record and rollback deletes the file.
~/.claude/ plugins/ marketplaces/	—	Created	mkdir -p. Not recorded in the manife
~/.claude/ plugins/ marketplaces/ Sagargupta16/ claude-cost- optimizer/	—	Cloned once	From https://github.com/Sagargupta16/claude-cost-optimizer.git, only if th directory does not already exist. Fail swallowed. Not recorded.
~/.claude/ plugins/ marketplaces/ JCodesMore/ jcodesmore- plugins/	—	Cloned once	From https://github.com/JCodesMore/jcodesmore-plugins.git, same conditi recorded.

Path	Component	Action	Detail
~/.kimi-code/ mcp.json	kimi	Created / merged	configure_mcp_for_agent: snapshot, - absent, then .mcpServers += {lumen, codegraph}. Skipped entirely unless code exists. Backed up to ~/.kimi-code/mcp.json.bak.<ts>. Recorded CREATED the wizard had to create it, so rollback removes it.
~/.config/ opencode/ opencode.json	opencode	Merged	Opencode uses a .mcp object, not .mcpServers: .mcp.lumen = {"type": "local", "command": [<wrapper>, "stdio"], "enabled": true} CodeGraph is not added here. Skipped the file already exists — the wizard create an Opencode config. Backed up to ~/.config/opencode/opencode.json.bak.
~/.qwen/ settings.json	qwen	Created / merged, by two different steps	Step 5's configure_mcp_for_agent does same .mcpServers merge as Kimi and skipped entirely unless ~/.qwen exists 3's configure_telemetry_optout separately sets .usageStatisticsEnabled = false when the file already exists and only is not already false. Both back it up to ~/.qwen/settings.json.bak.<ts>; the keeps one row, from whichever ran first (3).
(MiMo Code)	—	Nothing MiMo- specific is written	~/.mimocode exists, but MiMo inherits servers from ~/.claude.json (it labels claude:~/.claude.json) and also ships mcp CLI. The ~/.claude.json row about what configures it. The wizard probes timeout 25 mimo mcp list and reports honestly either way. The earlier claim MiMo “exposes no documented MCP file” and needed a manual step was tests A45, A46.
~/.bun/bin/	—	Created by the Bun installer	Only when Bun is installed by the wizard prepends this to PATH for the current session only — it does not write it into any settings and does not record it.
~/.claude/ plugins/cache/	—		curl -fsSL https://plugin.ashlr.ai/install.sh \& bash, run only when bun

Path	Component	Action	Detail
ashlr-marketplace/ashlr/		Cloned by the ashlr installer	available and the directory does not exist. It creates no binary and does not touch settings.json — that is by design which is why check_command ashlr could never succeed and was removed (tested). Not recorded in the manifest. Note that it is hardcoded to \$HOME/.claude, unlike the ACTION_REQUIRED detection which honors CLAUDE_CONFIG_DIR.
~/.ashlrcode/ (none)	—	Not created by the wizard	The genome lives at <project-root>/.ashlrcode/genome/ and is created by ashlr:ashlr-genome-init inside Claude not by any script.
~/.local/share/lumen/	—	Created by the Lumen CLI, not the wizard	Where lumen index stores its index data, one directory per project index. lumen deletes them.
~/.codegraph/telemetry.json	—	Written by codegraph telemetry off	Step 3's telemetry opt-out invokes the CodeGraph CLI, which persists the settings here. Not recorded in the manifest — the wizard never writes the file itself. The summary reads .enabled from it.

Paths that are no longer written. ~/.claude/mcp.json, ~/.kimi/config.json, ~/.opencode/config.json, ~/.mimo/config.json and ~/.qwen-code/config.json were written by an earlier revision. None of them is read by the agent it was meant for — they were orphan files that still earned a green tick in the summary. If they exist on your machine they are leftovers; delete them.

backup_file produces a **new** .bak.<ts> on every run for every file that already exists, *and* a manifest row. Prune the .bak siblings yourself; nothing rotates them, and scripts/rollback-agents-wizard.sh does not remove them either.

Project root (PROJECT_ROOT)

Path	Action
submodules/	mkdir -p
submodules/superspec/	

Path	Action
	Submodule init, or clone, per setup_superspec. Never deleted when it is a real git checkout.
.specify/ and/or specs/	Created by specify init --force (falling back to specify init -y), only if neither already exists
MANUAL-STEPS.md	Written on every run, unconditionally, from the manual-step registry. Overwrites the previous copy. Not recorded in the manifest, so rollback-agents-wizard.sh will not remove it — rm it yourself, or add it to .gitignore. See Step 9
.codegraph/	Created by codegraph init "\$PROJECT_ROOT" in Step 8 only , i.e. only when WIZARD_INDEX_PROJECT is set and no .codegraph/codegraph.db exists yet. On later runs codegraph sync updates it in place. Not recorded in the manifest
.ashlrcode/genome/	Not created by the wizard. Created by /ashlr:ashlr-genome-init inside Claude Code. The wizard only <i>detects its absence</i> , to decide whether to raise the genome manual step
.lumen-reindex.log	Not created by the wizard. Appended by scripts/lumen-reindex.sh; override with LUMEN_REINDEX_LOG
.test-evidence/<UTC>/	Not created by the wizard. Written by scripts/test-setup-agents-wizard.sh

The wizard also runs specify extension add ./submodules/superspec --dev when both specify and the path are present. Failures are swallowed.

Step 8 — project indexing (opt-in)

The whole step is wrapped in `if [[ -n "${WIZARD_INDEX_PROJECT:-}" ]]`. Unset, the wizard prints `Skipping project indexing (set WIZARD_INDEX_PROJECT=1 to build indexes).` and the Step 8 header never appears. Set, it runs two independent halves against `PROJECT_ROOT`:

Condition	Command	Note
codegraph on PATH, \$PROJECT_ROOT/.codegraph/ codegraph.db exists	<code>codegraph sync "\$PROJECT_ROOT" 2>&1 \ tail -3</code>	Incremental. codegraph index is never used — it discards the existing database before rebuilding. Test A27 asserts the literal <code>codegraph index</code> is absent, A26 that <code>codegraph sync</code> is present
codegraph on PATH, no database yet	<code>codegraph init "\$PROJECT_ROOT" 2>&1 \ tail -3</code>	Creates <code>.codegraph/</code> in the project root
codegraph missing	—	<code>codegraph</code> not installed - skipping its index.
lumen on PATH	<code>lumen index "\$PROJECT_ROOT" 2>&1 \ tail -3</code>	Incremental; needs a reachable embedding backend
lumen missing	—	<code>lumen</code> not installed - skipping its index.

Only the last three lines of each tool’s output are shown. Neither index is recorded in the rollback manifest: remove them with `rm -rf .codegraph` and `lumen purge "$PROJECT_ROOT"`. Tests **A25–A27** cover the opt-in flag and the sync-not-index rule.

Global npm and installers

Only three npm packages are installed, each only when its command is missing, each recorded as an ACTION row so rollback can undo it:

Command	Package	Component	Recorded undo
codegraph	@colbymchenry/ codegraph	npm	npm uninstall -g @colbymchenry/ codegraph
glyphdown	glyphdown	npm	npm uninstall -g glyphdown
qwen	@qwen-code/ qwen-code	npm	npm uninstall -g @qwen-code/ qwen-code

specify comes from `uv tool install specify-cli` when `uv` is available — a Python tool, not an npm package. It **does** record an undo action, under its own component: `record_action speckit "uv tool uninstall specify-cli"`, written only when `specify` is on PATH after the install. So `./scripts/rollback-agents-wizard.sh --run-actions` will uninstall SpecKit; scope it with `-c npm` if you want the npm packages gone but SpecKit kept.

The installs that record **nothing** are Bun (<https://bun.sh/install>), ashlr (<https://plugin.ashlr.ai/install.sh>) and WOZCODE (whatever `WOZCODE_INSTALL_CMD` contains). All three write outside the project through third-party installers.

Kimi, Opencode and MiMo Code are **detected, never installed**. Each ships its own installer, and the wizard prints the vendor URL when the command is missing:

Agent	Command	Vendor
Kimi	kim	https://kimi.moonshot.cn
Opencode	opencode	https://opencode.ai
MiMo Code	mimo	https://github.com/XiaomiMiMo

Package names that must never come back. Installing by guessed name either 404s or pulls in unrelated software, and the old `|| true` reported both as success:

Name once used	Reality
@qoomon/lumen	404. Lumen is a Claude Code plugin binary, set up in Step 3
@monday/codegraph	404. The real package is @colbymchenry/codegraph
@specify/cli	404. The real tool is specify-cli, installed with uv tool
opencode	404. Ships its own installer into ~/.opencode/bin
kimi	Exists , but it is a state-animation library — not the Kimi agent
mimo	Exists , but it is a minimal mobile build tool — not MiMo Code
qwen-code	As an npm package : a v0.0.5 stub — the real package is @qwen-code/qwen-code. As a command name : nothing installs it; @qwen-code/qwen-code provides qwen

Tests **A2**, **A12**, **A13**, **A14**, **A15**, **A16** and **A17** assert that no executable line reintroduces any of them and that the real names and the `npm_install_if_missing` guard are present.

Managed blocks

Three blocks are written across two files — two in `~/.bashrc`, one in `~/.bash_profile` — each delimited by literal marker lines that must begin at column 1.

`~/.bashrc` — Lumen block

```
# >>> lumen semantic search (managed by Claude Code) >>>
# <<< lumen semantic search <<<
```

Written by `configure_lumen_shell`. Contents: a `PATH` guard that prepends `$HOME/.local/bin` only when it is not already present; a fallback that sources the completion file when `bash-completion's _completion_loader` is unavailable; and six **commented-out** export

lines documenting the tuning knobs. The guard is written with `$PATH` and `$HOME` unexpanded, so the file never hardcodes the installing user's home directory — test **C7** asserts this.

~/.bashrc — telemetry opt-out block

```
# >>> telemetry opt-out (managed by Claude Code) >>>
# <<< telemetry opt-out <<<
```

Written by `configure_telemetry_optout`, which runs *before* the Lumen block (it is the first thing `ensure_lumen` does), so this block appears above it in the file. Contents, in full:

```
# Opt out of usage analytics for locally installed developer
  tooling.
# DO_NOT_TRACK is the cross-tool convention
  (consoledonottrack.com).
export DO_NOT_TRACK=1
export CODEGRAPH_TELEMETRY=0
```

`DO_NOT_TRACK` is the cross-tool standard and is read by `@colbymchenry/codegraph`; `CODEGRAPH_TELEMETRY` is CodeGraph's own switch. Both were verified against the installed package rather than guessed, and both are also exported into the wizard's running process so the rest of the run is covered. `WIZARD_KEEP_TELEMETRY=1` suppresses the whole block. Test **H1** asserts exactly one block, **H2** that a second run does not add another, and **H4** that sourcing the file really yields `DO_NOT_TRACK=1 / CODEGRAPH_TELEMETRY=0`.

Two telemetry settings live outside the shell and so are not part of any block: `codegraph telemetry off` (persisted by CodeGraph in `~/.codegraph/telemetry.json`) and `.usageStatisticsEnabled = false` in `~/.qwen/settings.json`. **Lumen ships no telemetry at all** — its binary contains no analytics strings — so nothing is set for it, and the final summary says so rather than implying a switch exists.

~/.bash_profile

```
# >>> user bin on PATH (managed by Claude Code) >>>
# <<< user bin on PATH <<<
```

Contents: the same `PATH` guard, and nothing else. Both files are needed for different reasons. Login shells start from the `PATH` set by `/etc/profile`, and `~/.bashrc` returns early for non-interactive shells — so without the `~/.bash_profile` block, `lumen` and every other `~/.local/bin`

tool would be missing from `bash -lc, ssh host '<cmd>', cron jobs` and `systemd user units`. That matters here because agents spawn MCP servers from exactly those non-interactive contexts.

Uninstalling cleanly

The supported way to undo a run is to replay its manifest: `./scripts/rollback-agents-wizard.sh --component shell --dry-run, then --yes`. That restores both files byte-exactly, markers and all. Everything below is the by-hand fallback for when the manifest is gone — see [SAFETY-AND-ROLLBACK.md](#).

Delete each block, markers included. `sed` is awkward because the markers contain regex metacharacters, so use the same `awk` approach the wizard itself uses:

```
strip() {
  awk -v s="$2" -v e="$3" '
    index($0, s) == 1 { skip = 1 }
    !skip                { print }
    index($0, e) == 1 { skip = 0 }
    ' "$1" > "$1.tmp" && cat "$1.tmp" > "$1" && rm -f "$1.tmp"
}

strip ~/.bashrc      '# >>> lumen semantic search (managed by
                    Claude Code) >>>' \
                    '# <<< lumen semantic search <<<'
strip ~/.bashrc      '# >>> telemetry opt-out (managed by Claude
                    Code) >>>' \
                    '# <<< telemetry opt-out <<<'
strip ~/.bash_profile '# >>> user bin on PATH (managed by Claude
                    Code) >>>' \
                    '# <<< user bin on PATH <<<'
```

catting the temp file back over the original (rather than `mv`) preserves the file's permissions and inode — test **C9** asserts that a `600 ~/.bashrc` stays `600`.

The rest of the uninstall:

```
lumen purge          # drop every index under
                    ~/.local/share/lumen/ (do this FIRST)
rm -f ~/.local/bin/lumen
rm -f ~/.local/share/bash-completion/completions/lumen

claude mcp remove lumen -s user # Claude Code: via its own CLI,
                                into the ACTIVE_CLAUDE_CONFIG_DIR
tmp=$(mktemp) && jq 'del(.mcpServers.lumen)' ~/.claude.json >
"$tmp" && mv "$tmp" ~/.claude.json
```

*# ^ the mirror; MiMo and other
inheritors read this file*

```
tmp=$(mktemp) && jq
  'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.kimi-
  code/mcp.json > "$tmp" && mv "$tmp" ~/.kimi-code/mcp.json
tmp=$(mktemp) && jq
  'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.qwen/
  settings.json > "$tmp" && mv "$tmp" ~/.qwen/settings.json
tmp=$(mktemp) && jq
  'del(.mcp.lumen)' ~/.config/
  opencode/opencode.json > "$tmp" && mv "$tmp" ~/.config/
  opencode/opencode.json
```

The Glyphdown hook lives in .hooks.PreToolUse and .hooks.PostToolUse as array entries, so remove the glyphdown ones and keep your own:

```
tmp=$(mktemp) && jq '
  def strip: (. // []) | map(select((.[hooks[]]?command) |
    index("glyphdown")) | not));
  .hooks.PreToolUse |= strip | .hooks.PostToolUse |= strip
' ~/.claude/settings.json > "$tmp" && mv "$tmp" ~/.claude/
  settings.json
```

The telemetry settings are the other half. The ~/.bashrc block is removed by the third strip call above; the two out-of-band settings are undone separately:

```
tmp=$(mktemp) && jq 'del(.usageStatisticsEnabled)' ~/.qwen/
  settings.json > "$tmp" \
  && mv "$tmp" ~/.qwen/settings.json # back to the Qwen
  default
codegraph telemetry --help # CodeGraph's own
  switch; the wizard only ever runs `off`
```

Every one of those files has a .bak.<ts> copy next to it, and a byte-exact copy inside the rollback session, if you would rather restore than edit.

Environment variable reference

Read by the wizard

Variable	Used by	Effect
WIZARD_STATE_DIR	backup_init, and scripts/ rollback-agents-wizard.sh	Root of the rollback state. Default \$HOME/.local/share/ setup-agents-wizard; sessio land in \$WIZARD_STATE_DIR/

Variable	Used by	Effect
		backups/<UTC>/.
		Resolved a call time inside backup_in
		so exporting it after sourcing
		the wizard in library mode
		still takes effect. The rollback
		tool reads the same variable
		so set it identically for both
		it will report No backup
		sessions found.
		eval'd to install WOZCODE.
		Unset → the step is skipped
		with a warning. Runs with
		your privileges; set it only
		command you trust.
		Overrides the model the
		wizard pulls, checks for, and
		names in the commented
		~/ .bashrc line. Default order
		jina-embeddings-v2-base-c
		The base URL probed with
		curl -sf --max-time 5
		"\$OLLAMA_HOST/api/tags".
		Default http://
		localhost:11434.
		Any non-empty value stops
		execution before Step 1.
		Any non-empty value makes
		the function return
		immediately with
		WIZARD_KEEP_TELEMETRY set
		leaving telemetry settings
		alone.: no ~/ .bashrc block,
		codegraph telemetry off, no
		~/ .qwen/settings.json edit
		The summary then prints
		left untouched on request
		(WIZARD_KEEP_TELEMETRY).
		Unset (the default) the opt-
		is applied. Test H5 .
		Any non-empty value skips
		only the Glyphdown hook
WOZCODE_INSTALL_CMD	Step 2	
LUMEN_EMBED_MODEL	Constant block, ensure_ollama, final summary	
OLLAMA_HOST	verify_lumen	
SETUP_WIZARD_LIB_ONLY	Library-mode guard	
WIZARD_KEEP_TELEMETRY	configure_telemetry_optout (Step 3), final summary	
WIZARD_SKIP_GLYPHDOWN_HOOK	Step 5, final summary	

Variable	Used by	Effect
		registration: WIZARD_SKIP_GLYPHDOWN_HOOK is set - glyphdown hook M registered. plus Enable it later by re- running without that variable. Glyphdown is sti installed in Step 2, and ~/.claude/settings.json is not touched at all. The summary prints Glyphdown Hook skippe on request (WIZARD_SKIP_GLYPHDOWN_HO The hook fires on every too call, so this exists to let you defer wiring an unvetted binary into a running sessi Test A19 requires the opt-o to exist. Any non-empty value enab <u>Step 8</u> : codegraph sync/ codegraph init plus lumen index for PROJECT_ROOT. Uns (the default) the step head never prints. Tests A25–A2 and A10 which asserts the step headers still read 1,2,3,4,5,6,7,8,9,. Any non-empty value skips the closing read that waits Enter after the ACTION REQUIRED section. The secti still prints and MANUAL - STEPS.md is still written. On relevant on a TTY — the pa is already skipped when [[-t 0]] is false. Read, never written. Defa to \$HOME/.claude. Determin where the wizard looks for plugins/cache/ashlr-
WIZARD_INDEX_PROJECT	Step 8	
WIZARD_NONINTERACTIVE	<u>Step 9</u>	
CLAUDE_CONFIG_DIR	CLAUDE_DIR constant, used in <u>Step 9</u>	

Variable	Used by	Effect
		marketplace/ashlr/ and plugins/marketplaces/ashlr/ marketplace/ when deciding whether the ashlr steps are still pending. It is also where claude mcp add-json -s us really writes, which is the reason for the ~/.claude.json mirror. Note that Step 2's a install check and the final summary hardcode \$HOME/.claude and do not honour this variable.

The three operational scripts read their own:

Variable	Read by	Default	Effect
MAX_ROUNDS	lumen-reindex.sh	40	Retry rounds before giving up (exit 1)
LUMEN_REINDEX_LOG	lumen-reindex.sh	<project>/.lumen-reindex.log	Where the run log is appended
LUMEN_STORE	lumen-index-doctor.sh	\$HOME/.local/share/lumen	Root the doctor globs for */index.db
LUMEN_EMBED_MODEL	ollama-vulkan-remediation.sh, lumen-reindex.sh	ordis/jina-embeddings-v2-base-code	Model used by the batch probes
OLLAMA_HOST	ollama-vulkan-remediation.sh, lumen-reindex.sh	http://localhost:11434	Backend endpoint

Full reference: [OPERATIONAL-SCRIPTS.md](#).

Read by the wrapper

Variable	Effect
LUMEN_BIN	Absolute path to a Lumen binary. When set, <code>~/local/bin/lumen</code> execs it directly and skips all glob resolution. If set but not executable, the wrapper exits 127 with a diagnostic. Read by the wrapper only — the Lumen binary itself does not consult it.

Read by the Lumen CLI

Every name below is present in the Lumen binary. The wizard writes all of them into `~/bashrc` **commented out** and never exports them.

Variable	Documented by	Meaning
LUMEN_BACKEND	wizard comment; <code>--backend</code> flag help	Embedding backend selector: <code>ollama</code> or <code>lmstudio</code> .
LUMEN_EMBED_MODEL	<code>lumen index --help</code>	Embedding model. <code>--help</code> states the default is <code>\$LUMEN_EMBED_MODEL</code> or <code>ordis/jina-embeddings-v2-base-code</code> .
LUMEN_EMBED_CTX	binary only	Embedding context size. Not documented in <code>lumen --help</code> ; no CLI flag exposes it.
LUMEN_EMBED_DIMS	binary only	Embedding vector dimensionality. Not documented in <code>lumen --help</code> ; no CLI flag exposes it.
LUMEN_FRESHNESS_TTL	wizard comment	Seconds before an index is treated as stale. Wizard

Variable	Documented by	Meaning
		comment suggests 300.
LUMEN_LOG_LEVEL	wizard comment	debug info warn error.
LUMEN_MAX_CHUNK_TOKENS	binary only	Upper bound on tokens per indexed chunk. Not documented in <code>lumen --help</code> ; no CLI flag exposes it.
LUMEN_REINDEX_TIMEOUT	wizard comment	Seconds allowed for a re-index; raise for very large repositories. Wizard comment suggests 600.
OLLAMA_HOST	binary; wizard	Where the CLI reaches the Ollama daemon. Default <code>http://localhost:11434</code> .

Why the wizard leaves them unset. The CLI you type at a prompt and the MCP servers your agents spawn share the same built-in defaults. Pinning a value in `~/.bashrc` desyncs them the moment a plugin update changes a default — and because the embedding model is part of an index's identity, a model mismatch makes Lumen build a **second index per project**: double the indexing time, double the disk, and searches that miss content sitting in the other index. Uncomment a line only when you have a specific reason, and run `lumen purge` afterwards so nothing orphaned is left behind.

Library mode (SETUP_WIZARD_LIB_ONLY)

Everything above the guard — all colors, all helper functions, all Lumen constants, `setup_superspec` — is pure definitions. Setting `SETUP_WIZARD_LIB_ONLY` to any non-empty value makes the script return 0 (or exit 0 if it was executed rather than sourced) immediately before `print_header "AI Agents Ultimate Setup Wizard"`:


```
if [[ -n "${SETUP_WIZARD_LIB_ONLY:-}" ]]; then
    return 0 2>/dev/null || exit 0
fi
```

Note that `SCRIPT_DIR` and `PROJECT_ROOT` are computed *before* the guard, so they are available to sourcing code.

Use it to exercise one helper in isolation:

```
SETUP_WIZARD_LIB_ONLY=1 . scripts/setup-agents-wizard.sh
install_lumen_wrapper
echo "$LUMEN_WRAPPER"
```

`scripts/test-setup-agents-wizard.sh` relies on this. Its `in_sandbox` helper runs a snippet against a throwaway `$HOME`, so nothing in the real environment is mutated:

```
in_sandbox() {
    local box; box=$(mktemp -d)
    HOME="$box" SETUP_WIZARD_LIB_ONLY=1 bash -c "
        set -uo pipefail
        . '$WIZARD'
        # The wizard sets 'set -euo pipefail' at its top, and
        # sourcing it leaks
        # errexit into this shell. Tests deliberately run commands
        # that exit
        # non-zero (127 checks), so errexit must be off or the
        # shell dies
        # before the assertion can report.
        set +e
        $1
    " 2>&1
    rm -rf "$box"
}
```

The `set +e` matters: without it the sourced wizard's `set -euo pipefail` would kill the sandbox shell on the first deliberately-failing command, before the assertion could record anything.

Lumen CLI reference

Confirmed against `lumen 0.0.41`. Invoke through the wrapper at `~/.local/bin/lumen`.

Usage:

```
lumen [command]
```

Available Commands:

```
completion  Generate the autocompletion script for the specified
```

```

shell
  help          Help about any command
  hook          Hook handlers for AI coding agent integration
  index         Index a project for semantic search
  purge         Remove lumen index data
  search        Search an indexed project for semantically similar
code
  stdio         Start the MCP server on stdin/stdout
  version       Print the lumen version

```

There is **no serve subcommand**. `lumen serve` fails with
 Error: unknown command "serve" for "lumen".

lumen index <project-path>

Indexes a project for semantic search. **Incremental** — re-running processes only what changed, so it is cheap to repeat after large external edits. Requires a reachable embedding backend; without one it cannot chunk or embed anything.

Flag	Description
-b, --backend <string>	Embedding backend to select (ollama or lmstudio); disambiguates when --model is configured on multiple backends
-f, --force	Force a full re-index instead of an incremental one
-m, --model <string>	Embedding model (default: \$LUMEN_EMBED_MODEL or ordis/jina-embeddings-v2-base-code)
-h, --help	Help for index

lumen search <query> [flags]

Searches an already-indexed project for semantically similar code.

Flag	Description
-b, --backend <string>	Embedding backend to select (ollama or lmstudio)
--cwd <string>	Project root, for when --path is a subdirectory of it
-f, --force	Force a full re-index before searching

Flag	Description
<code>--max-lines <int></code>	Truncate snippets at N lines (0 = unlimited)
<code>--min-score <float></code>	Minimum score threshold, -1 to 1
<code>-m, --model <string></code>	Embedding model override
<code>-n, --n-results <int></code>	Max results to return (default 8)
<code>-p, --path <string></code>	Directory to search (default: cwd)
<code>--summary</code>	Omit code snippets, return locations only
<code>--trace</code>	Print per-phase timing to stderr
<code>-h, --help</code>	Help for search

lumen purge [path...]

Deletes index databases under `~/.local/share/lumen/`.

- With **no arguments**, removes every index. Irreversible — all indexes rebuild on the next search.
- With **one or more paths**, removes only those projects' index directories. Each path is normalized to its git root first, then matched against the `project_path` recorded inside each index database, so switching embedding models or using custom models never leaves orphan indexes.
- Indexes created by older binaries that did not record `project_path` cannot be matched by path; wipe those with a bare `lumen purge`.
- A concurrently running indexer for a purged project may log a write error and exit; re-run `lumen index` afterwards to rebuild.

Only `-h, --help`.

lumen stdio

Starts the MCP server on stdin/stdout. This is what every agent config the wizard writes invokes. Not meant to be run by hand. Only `-h, --help`.

lumen completion <shell>

Generates the autocompletion script. Shells: bash, fish, powershell, zsh. The wizard uses `lumen completion bash` and writes the result to `~/.local/share/bash-completion/completions/lumen`. Test **F5** asserts the generated bash script registers `__start_lumen`.

lumen version

Prints the version string alone, e.g. `0.0.41.verify_lumen` uses it as the launcher’s liveness probe.

lumen hook <command>

Hook handlers for AI coding agent integration.

Subcommand	Description
pre-tool-use	Intercept Grep calls and suggest semantic search when appropriate
session-start	Output SessionStart hook JSON for Claude Code or Cursor

Test suite

`scripts/test-setup-agents-wizard.sh --no-live` skips the network and daemon checks. Evidence lands in `.test-evidence/<UTC timestamp>/` as `results.tsv`, `run.log` and `summary.json`. Exit status is non-zero if any assertion failed.

Ten groups, A–J. The file declares group `H` **before** group `G`, so the console output order is A, B, C, D, E, F, H, G, I, J.

Deriving the assertion count

Do not hardcode a total — it moves whenever a test is added, and it differs between a live run and `--no-live`. The current per-group record counts:

Group	Ids	Records	Note
A	50	54	

Group	Ids	Records	Note
			A1–A50; A12 is a loop over 5 bogus package names, so it emits 5 records. A11 skips without shellcheck
B	8	8	
C	10	10	
D	5	5	all 5 become skips without jq
E	3	3	
F	9	11 live / 7 with --no-live	F2 is a loop over 3 shell kinds; --no-live records F1–F7 as skips, and F5b/F8 are not recorded at all in that mode
G	18	18	G16–G18 skip without jq
H	7	7	H6/H7 skip without jq
I	14	14	I4–I14 are 11 assertions; I1–I3 are a 3-iteration loop over the three operational scripts
J	8	8	J2–J7 each build and tear down a throwaway git repository

Full live run: **138**. --no-live: **134**. **Neither number belongs in a bug report** — quote summary.json’s {total, passed, failed, skipped} for the run you actually did. Three groups emit more records than they have ids (A, F, I), which is why an id count and a record count are not the same thing.

Group	Coverage
A	Static analysis, A1–A50: bash -n, absence of every bogus package name, presence of the real ones and of the npm_install_if_missing guard, stdio not serve, absolute MCP command, the -e gitlink test, qwen rather than qwen-code as the probed binary, PreToolUse present and "ToolCall" absent, the WIZARD_SKIP_GLYPHDOWN_HOOK / WIZARD_INDEX_PROJECT / WIZARD_KEEP_TELEMETRY switches, codegraph sync present and codegraph index absent, uv tool uninstall specify-cli recorded, export DO_NOT_TRACK=1, no jq // on a boolean, the /api/embed health round-trip and its wedged-NaN branch, library-mode guard, sequential step headers 1..7, shellcheck -S error. A35–A41: /api/ps GPU-residency probe, the inference compute log line, the Vulkan verdict gated on the value actually read, an unreadable library reported as unknown, the runbook pointer, “never names a backend it did not probe”, and A41 — the wizard carries no systemctl restart, no /etc write and no sudo in this path. A42–A50: ashlr not probed as a CLI and verified by plugin directory instead, the ~/.claude.json mirror, the stale “no documented MCP config”

Group	Coverage
	claim gone, MiMo probed with mimo mcp list, every lumen search probe timeout-bounded, the ACTION REQUIRED section, MANUAL-STEPS.md, and the plugins/marketplaces/ activation probe
B	Wrapper unit tests in an isolated \$HOME: executability, sort -V version selection, ~/.claude* fallback, LUMEN_BIN override, exit 127 on a bad or missing binary
C	Shell config: exactly one block per file, idempotence across three runs, generated files parse as bash, unexpanded \$PATH guard, ~/.local/bin added exactly once, mode 600 preserved, pre-existing content preserved
D	MCP config: args == ["stdio"], absolute command, valid JSON, foreign keys preserved, exactly two servers after two runs
E	SuperSpec data-loss regression: a gitlink submodule survives; a stale non-git directory is still replaced
F	Live: lumen on PATH across login/ interactive/non-interactive shells, backend reachable, model present, completion registration, and a true end-to-end index- then-search against a two-file fixture repository
G	Backup manifest and rollback, G1-G18, in a throwaway \$HOME with WIZARD_STATE_DIR pointed inside it: manifest header, MODIFIED vs CREATED, the stored copy holds the original content

Group	Coverage
	and mode 600, first-snapshot-wins, byte-exact restore, deletion of a created file, --dry-run inertness, --component isolation, the pre-rollback snapshot, ACTION reported but not executed, --list. G16–G18 are the snapshot-ordering regression: a brand-new agent config is recorded CREATED, rollback deletes it instead of leaving an empty {}, and a pre-existing config still restores to its true original (these three skip without jq) Telemetry opt-out, H1–H7, in a throwaway \$HOME: exactly one #>>> telemetry opt-out ... block, still one after a second run, the generated ~/.bashrc parses as bash, sourcing it exports DO_NOT_TRACK=1 / CODEGRAPH_TELEMETRY=0, WIZARD_KEEP_TELEMETRY writes no block at all, .usageStatisticsEnabled is set to false in ~/.qwen/settings.json without dropping existing keys, and an already-false value is detected as already disabled rather than “unset” (H6/H7 skip without jq)
H	
	Operational scripts, I1–I14: each of the three is executable and parses (I1–I3, one record each); the remediation script offers --check (I4) and --rollback (I5) and defaults to the read-only action (I6, matching the literal case "\${1:---check}"); its probe tests aggregate distinctness (I7); the doctor checks duplicate-vector groups
I	

Group	Coverage
J	(I8), still runs the per-vector NaN/all-zero tests (I9) and opens the database mode=ro (I10); the reindexer refuses to start on a Vulkan backend (I11), --force is proven to actually reach lumen index -f rather than merely being parsed (I12), and the reason --force is required after corruption is documented in the script itself (I13); neither the reindexer nor the doctor calls sudo (I14). Group I only reads source text Hardcoded-path audit, J1–J8: scripts/audit-hardcoded-paths.sh is executable and parses (J1); then it is <i>exercised</i> against throwaway git repositories rather than grepped — clean repo exits 0 (J2), a machine-specific path exits 1 (J3), a comment describing the historical bug does not count as a violation (J4), \$HOME and ~ are not flagged (J5), .hardcoded-paths-allow really suppresses a listed file (J6), and /etc /usr /tmp are not treated as machine-specific (J7). J8 applies the rule to this repository’s own scripts/*.sh with no exemptions. Note the test file builds its bad-path literals from concatenated fragments, so that the file testing the rule does not itself violate it